Problem Statement/Area of Investigation

AI coding assistants have become the standard. In Stack Overflow's 2025 survey of over 49,000 developers, 84% were using or planning to use AI tools, and half of professionals used them every day. The term "vibe coding", meaning using AI prompted by natural language to help write code, was coined by Andrej Karpathy and became Collins' word of the year for 2025.

Vibe coding has real benefits, chiefly that it makes coding more accessible. The key risk is understanding. In a randomised trial of 52 developers learning an unfamiliar Python library, those using AI scored 50% on a comprehension test, against 67% for those who coded by hand, with the largest gap in debugging, a key part of coding. A PNAS field experiment with nearly a thousand students found that unrestricted access to GPT-4 led to grades 17% lower than students who never had it, once the AI was taken away. That suggests the problem is not unique to coding.

Importantly, how AI is used matters. In both studies, designs that kept learners engaged largely avoided the harm. That is the gap Phronesis is built to fill.

The Key Flaw with Current Solutions

So, what exists today?

Interview platforms like LeetCode offer a huge fixed bank of problems, built around passing interviews. Third-party AI add-ons now layer on weakness analysis and hints, but they're still aimed at interview candidates and still revolve around solving problems, rather than understanding how code works.

Course platforms like Codecademy offer AI tutoring, but within a set curriculum.

Research has traced learner progress for decades using knowledge tracing, but most models rely on quiz-style right or wrong data, which one review calls "insufficient to reflect a clear picture of students' learning process."

So the gap is adaptive practice aimed at comprehension: reading, tracing, and debugging code. In novice programmers, tracing and explaining skills are linked to the ability to write code.

Proposed AI Solution

Phronesis is a daily practice tool that adapts to what you struggle to understand. Each session gives you a short chain of tasks on one piece of code: fix the bug, refactor it into a class, reason about its complexity, then extend it.

There are three AI components. First, a learner model using Bayesian knowledge tracing, which keeps a mastery estimate for each concept and skill and updates it after every attempt. It is lightweight, interpretable, and has a long track record in intelligent tutoring. Second, an LLM generates the code, tests and bugs at scale, and every task is validated by actually running it, because LLMs can state wrong answers confidently. Third, the LLM provides tiered hints that nudge you without giving the answer, because the evidence we saw shows engaged use of AI protects learning.

The learner model then decides what you practise next, so effort goes where understanding is weakest.

Proof of Concept

Here is how it fits together.

Offline, an LLM generates the code, tests and hints, and a validator runs everything before it enters the question bank. When a learner starts a session, the selector picks tasks targeted towards the learner's weakest skills.

They work in the UI, can ask the hint model for up to three levels of help, and their code runs in a sandbox against the ground truth.

The result updates the learner model, which stores mastery per skill, feeding straight back into what gets selected next.

This means the AI is sitting in three unique places: generation, hints and the learner model.

(switch tab to terminal to run python as example. do example run of using hints, answering right, answerign wrong, quitting to show score)

Alternative Solution Comparison

There is a simpler alternative: a curated question bank with rule-based tagging. Each question labelled and fixed rules would select the next task based on previous mistakes the learner made.

This would be cheaper, easier to validate and more predictable, but it would require substantial manual authoring and offers less flexibility

We trade that simplicity for more granular mastery tracking, the ability to generate additional validated tasks and have the competitive advantage of not just "using" ai, but combining many different things in order to have one feedback loop.

External Obstacles

Economically, repeated LLM calls could make daily practice expensive, so I would measure cost per session, and use smaller models and/or caching where appropriate.

Technologically, the most important risk is incorrect AI-generated "things". My approach is to execute and validate questions before learners see them, although this can't eliminate every educational eror.

Learners might either distrust some feedback, or become overly dependent on hints, so I would tailor the product to encourage independent reasoning, and trying harder before giving up.

Finally, data protection and copyright must be considered before deployment. The prototype should minimise personal data, use appropriately licensed content and undergo a legal review before any wider release.

Internal Obstacles

Internally, I used a qualitative risk register to assess likelihood, impact and mitigation. This is consistent with the NIST AI Risk Management Framework's emphasis on assessing and prioritising risks, then allocating resources to manage them.

My highest-priority concern is question quality. If generated code or its expected output is wrong, the learner could be marked incorrectly, undermining trust in the entire system. Automated execution and test validation are therefore essential, with additional review for cases automation cannot reliably detect.

The learner model also has a cold-start problem because it has little evidence about a new user. A short diagnostic assessment can provide an initial estimate before the system adapts to subsequent attempts.

Finally, development time, API costs and learner retention could limit viability. I'd manage these by building the core learning loop first, controlling usage and testing short practice sessions with real learners before expanding the feature set.

Business case

I'd initially develop Phronesis as an internal learning pilot rather than launching it immediately as a commercial product. This would allow me to test whether adaptive practice actually improves code comprehension before committing to a larger investment.

For learners, the intended value is stronger debugging, code tracing and reasoning skills, rather than simply producing working code. For an organisation, Phronesis could provide more targeted learning support and visibility into which skills employees struggle with.

Its competitive advantage is the combination of mastery tracking, validated code tasks and graduated hints within one feedback loop.

If the pilot demonstrates measurable learning improvements, the product could expand into interview preparation and cybersecurity challenge practice, before potentially being offered to organisations as a corporate learning tool. These are future opportunities, though, not established revenue streams. The immediate business objective is to validate the learning outcomes and demand.

Business case: value

The resource requirements depend on whether we're running a small pilot or deploying a production service.

In the short term, the prototype can be developed by one person using Python, an existing language model API, a lightweight database and automated tests. The main costs would be development time, model usage and basic hosting. I would keep these controlled through usage limits and a small group of pilot learners.

At scale, we'd need additional software engineering and learning-design expertise, alongside secure code execution, authentication, monitoring and ongoing content validation. Hosting, inference and maintenance costs would also increase with usage.

This is why I'd build the adaptive learning logic and validation pipeline myself, but reuse existing language models and infrastructure. Training a foundation model from scratch would add substantial cost without being necessary to test the core idea.

Conclusion

To conclude, AI can make writing code easier without ensuring that learners understand what it does. Phronesis aims to bridge that gap through adaptive practice, validated tasks and guided hints. The prototype demonstrates the technical approach, but the next step is to test whether it genuinely improves comprehension, and whether those benefits justify the cost.